TECHNICAL ARCHITECTURE | FULLY FUNCTIONAL & VERIFIED | PRODUCTION READY

Smart Database Visualizer

Complete Technology Stack Specification, Implementation Mapping & Subsystem Deep-Dive

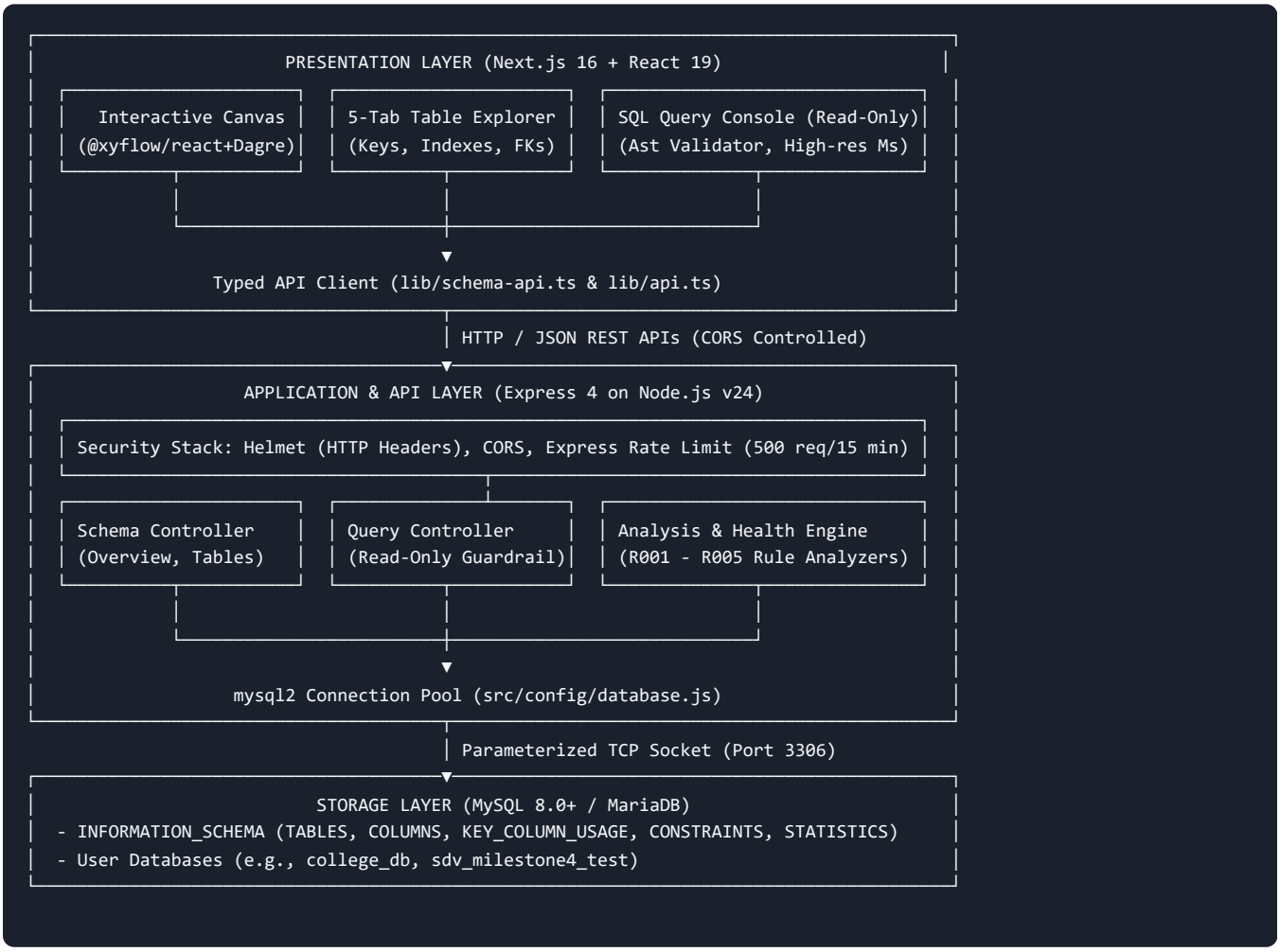
DOCUMENT VERSION	ARCHITECTURE TYPE	FRONTEND PORT	BACKEND PORT
Release 2.0 (Milestone 5+)	Decoupled Full-Stack REST	HTTP 3000 (Next.js 16)	HTTP 5000 (Express 4)

1. Executive Technical Summary

Smart Database Visualizer is a high-performance web platform that dynamically introspects relational MySQL databases, extracts their relational metadata via standard `INFORMATION_SCHEMA` catalogs with zero hardcoding, and renders an interactive, graph-theoretically arranged Entity-Relationship (ER) diagram in the browser.

The platform pairs an asynchronous **Next.js 16 (React 19)** frontend with a secured **Node.js + Express** backend. It provides automated layout (Dagre), complete five-tab schema exploration, AST-validated read-only SQL querying, heuristic database health analysis (rules R001–R005), and interactive CRUD operations.

2. End-to-End Architecture & Topology



Zero Hardcoding Principle

> No table names, column lists, or relationship links exist in application source code. All schema topology is introspected at runtime directly from the connected MySQL catalog.

3. Frontend Technology Stack & Implementation Mapping

The frontend is structured under `frontend/` and developed with the Next.js App Router paradigm.

TECHNOLOGY	VERSION	IMPLEMENTATION FILES	SPECIFIC SUBSYSTEM ROLE

Next.js	`16.3.3`	`frontend/app/page.tsx`
		`frontend/app/visualize/page.tsx`
		`frontend/next.config.mjs`
			App Router framework, client-side routing, Webpack production optimizer, and server/client state demarcation (`'use client'` directives).

React & DOM	`19.0.0`	`frontend/components/*`	Declarative UI component architecture, reactive hook state (`useState` , `useCallback` , `useMemo` , `useRef`), and virtual DOM rendering.

TypeScript	`5.7.3`	`frontend/lib/schema.ts`
		`frontend/lib/schema-api.ts`
		`frontend/tsconfig.json`
			Full strict type enforcement (0 compilation errors). Provides models for `DatabaseSchema` , `TableNode` , `ColumnDetail` , `QueryResult` , and `AnalysisIssue` .

@xyflow/react (React Flow)	`12.12.0`	`components/visualization/SchemaCanvas.tsx`
		`components/visualization/TableNode.tsx`
		`components/visualization/RelationshipEdge.tsx`
			Infinite-canvas graph rendering engine. Renders custom node wrappers with column badges and SVG cubic bezier relationship edges with direction arrows and interactive cardinality badges.

Dagre	`0.8.5`	`frontend/lib/schema/dagreLayout.ts`	Directed Acyclic Graph layout algorithm. Computes coordinate positions (x, y) for each table based on column counts, padding, and foreign key direction vectors (LR / TB).
Tailwind CSS	`4.3.3`	`frontend/app/globals.css` `frontend/postcss.config.mjs`	Modern utility styling with CSS theme variables (OKLCH color system), responsive breakpoints, and glassmorphic translucent surfaces.
Lucide React	`1.16.0`	Throughout frontend UI components	Vector iconography representing Database tables, Primary Keys (Key), Foreign Keys (Link), Indexes (Search), Terminal commands, and Warning indicators.
Base UI & Radix	`1.5.0`	`frontend/components/ui/modal.tsx`	Headless accessible UI primitives for accessible dialog focus traps, portaled overlays, and keyboard event navigation.
clsx & tw-merge	`Latest`	`frontend/lib/utils.ts`	Utility `cn()` function allowing dynamic class condition evaluation while deduplicating conflicting utility definitions.

4. Backend Technology Stack & Implementation Mapping

The backend resides in `backend/` and provides a robust, decoupled REST API built on Express.

TECHNOLOGY	VERSION	IMPLEMENTATION FILES	SPECIFIC SUBSYSTEM ROLE
Node.js	`v24.19.0`	`backend/src/server.js`	Asynchronous JavaScript execution environment running the micro-service architecture and connection pooling.

Express	`4.19.2`	`backend/src/server.js`
		`backend/src/routes/*`
		`backend/src/controllers/*`
			REST HTTP server managing routing, route handlers, middleware dispatch, JSON request parsing, and response status mapping.

mysql2 / promise	`3.10.1`	`backend/src/config/database.js`
		`backend/src/services/schemaService.js`
		`backend/src/services/dataService.js`
			High-performance MySQL driver utilizing native TCP connection pooling, parameterized queries (`?` placeholders), and Promise-based transaction execution.

Zod	`3.23.8`	`backend/src/validators/databaseValidator.js`
		`backend/src/validators/queryValidator.js`
		`backend/src/validators/schemaValidator.js`
			Schema input validation enforcing strict typing, regex safe identifier checks (`/^[a-zA-Z0-9_]+$/`), and port ranges before database hits.

Helmet	`7.1.0`	`backend/src/server.js (Line 31)`	HTTP security middleware configuring 11 protective HTTP headers against MIME sniffing, clickjacking, and XSS exploits.
CORS	`2.8.5`	`backend/src/server.js (Lines 34–40)`	Cross-Origin Resource Sharing control locking incoming requests to the configured frontend origin (e.g. `http://localhost:3000`).
express-rate-limit	`7.3.1`	`backend/src/server.js (Lines 43–53)`	DDoS protection middleware enforcing a ceiling of 500 requests per 15 minutes per IP address with standard rate limit headers.
Multer	`2.3.0`	`backend/src/controllers/importController.js`	Streaming multipart form-data parser supporting uploaded SQL schema dumps and dataset imports.
Nodemon	`3.1.4`	`backend/package.json (Line 8)`	Development process monitor enabling zero-downtime hot-reloading when backend source files are modified.

5. Database Layer & Catalog Introspection Engine

The visualizer does not rely on ORM models or static metadata files. It queries the MySQL `INFORMATION_SCHEMA` catalog in real time through parameter-bound queries:

1. Table Discovery Query

Discovers all user-created base tables in the active schema:

```
SELECT TABLE_NAME AS name
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = ?
  AND TABLE_TYPE = 'BASE TABLE'
ORDER BY TABLE_NAME;
```

2. Column Details Query

Fetches data types, nullability, defaults, and auto-increment flags:

```
SELECT COLUMN_NAME, DATA_TYPE,
       IS_NULLABLE, COLUMN_DEFAULT,
       COLUMN_KEY, EXTRA, COLUMN_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = ?
  AND TABLE_NAME = ?;
```

3. Foreign Key Relationship Query

Resolves referential integrity links between tables:

```
SELECT CONSTRAINT_NAME AS constraintName,
       TABLE_NAME AS sourceTable,
       COLUMN_NAME AS sourceColumn,
       REFERENCED_TABLE_NAME AS targetTable,
       REFERENCED_COLUMN_NAME AS targetColumn
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = ?
  AND REFERENCED_TABLE_NAME IS NOT NULL;
```

4. Index Metadata Query

Queries secondary, unique, and composite indexes:

```
SELECT INDEX_NAME AS indexName,
       COLUMN_NAME AS columnName,
       NON_UNIQUE AS nonUnique,
       SEQ_IN_INDEX AS seqInIndex,
       INDEX_TYPE AS indexType
FROM INFORMATION_SCHEMA.STATISTICS
WHERE TABLE_SCHEMA = ?
  AND TABLE_NAME = ?;
```

6. Core Feature Subsystems Deep-Dive

6.1 React Flow Visualization & Layout Engine

- **Custom Table Nodes (`TableNode.tsx`)**: Render table headers, primary key badges (`PK`), unique badges (`UQ`), index markers (`IDX`), data types, and nullable flags.
- **Custom Bezier Edges (`RelationshipEdge.tsx`)**: Verified database foreign keys render as **solid blue lines**. Heuristic recommendations (R003) render as **dashed amber lines** and are strictly isolated from the declared schema graph.
- **Dagre Layout Engine (`dagreLayout.ts`)**: Measures node heights dynamically based on column count ($\text{Height} = 42 + N \times 28\text{px}$) to compute collision-free, hierarchical coordinates in both Left-to-Right and Top-to-Bottom orientations.

6.2 SQL Query Console & AST Read-Only Protection

- **Single Endpoint**: Frontend calls `POST /api/query/execute` via `schema-api.ts`.
- **Strict Read-Only Enforcement (`queryService.js`)**: Allows only `SELECT`, `SHOW`, `DESCRIBE`, `DESC`, `EXPLAIN`, and read-only `WITH` CTE queries. Modifying statements (`INSERT`, `UPDATE`, `DELETE`, `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`) are rejected with error code `READ_ONLY_QUERY_CONSOLE`.

- **Multiple Statements Guard**: Semicolon-chained commands are parsed and rejected with `MULTIPLE_STATEMENTS_NOT_ALLOWED` to prevent batch injection.
- **Preserved SQLState & Error Codes**: Preserves native MySQL codes (e.g. `ER_NO_SUCH_TABLE`) and SQL states (e.g. `42S02`) cleanly to the client UI.

6.3 Health Inspection & Rule Engine

The backend analysis engine executes 5 specialized structural and relational health rules:

- **R001 — Missing Primary Key (HIGH)**: Detects tables lacking primary keys (1NF integrity violation).
- **R002 — Potential Orphaned Table (MEDIUM)**: Identifies standalone tables with zero incoming or outgoing foreign keys. Provides advisory *Review Guidance* without generating destructive SQL.
- **R003 — Possible Missing Foreign Key (MEDIUM)**: Matches naming heuristics (e.g. `customer_id` in unlinked tables) to suggest foreign key candidates. Rendered as non-binding dashed overlays.
- **R004 — Index Review Recommended (LOW)**: Detects unindexed join columns that cause full table scans.
- **R005 — Structural Dependency Analysis**: Builds a dependency DAG, identifying root, leaf, and circular dependencies.

Health Scoring Formula: $$\text{Score} = \max\Big(0, \, 100 - (15 \times \text{High} + 8 \times \text{Medium} + 3 \times \text{Low})\Big)$$

7. Foreign Key Pre-Validation Matrix (13-Point Verification)

Before executing `ALTER TABLE ... ADD CONSTRAINT`, both the client modal (`editor-modals.tsx`) and the server service (`schemaEditService.js`) execute a 13-point pre-validation check:

#	VALIDATION CRITERIA	VERIFICATION METHOD	ENFORCED ERROR RESPONSE
1	Source & Target Table Existence	Query `INFORMATION_SCHEMA.TABLES`	`Source/Target table '{name}' does not exist.`
2	Self-Referencing Identity Check	Compares table and column IDs	`Cannot create self-referencing FK on same column.`
3	Source & Target Column Existence	Query `INFORMATION_SCHEMA.COLUMNS`	`Source/Target column '{name}' does not exist.`
4	Datatype Family Compatibility	Compares base types (INT vs VARCHAR)	`Cannot create relationship: source {T1} and target {T2} are incompatible.`
5	Integer Precision Match	Compares INT, BIGINT, SMALLINT	`Cannot create relationship: source {T1} and target {T2} are incompatible.`
6	Integer Sign Match (Signed/Unsigned)	Checks `COLUMN_TYPE` for 'unsigned'	`Source column is {S} while target is {T}.`
7	Target Column Index / Key Check	Checks `INFORMATION_SCHEMA.STATISTICS`	`Target column must be indexed (e.g. PRIMARY KEY or UNIQUE).`
8	SET NULL Nullability Rule	Validates `IS_NULLABLE === 'YES'`	`SET NULL action requires source column to be nullable.`
9	Duplicate Relationship Check	Checks `KEY_COLUMN_USAGE`	`Foreign key relationship already exists.`
10	Constraint Name Collision Check	Checks `TABLE_CONSTRAINTS`	`Constraint name '{name}' already exists.`
11	Orphan Child Data Check	Subquery for child rows without parent	`Found {N} existing row(s) with orphan values.`
12	String Length & Collation Match	Checks character length & collation	Client badges & server verification.
13	Connection & Session State	Validates active pool allocation	`No database connected.`

8. Testing & Quality Assurance Verification Matrix

The system passes all **427 automated tests** across 9 verification suites:

TEST SUITE	TARGET SCOPE	RESULT
`backend/test-api.js`	Express API routing, 404 handlers, Zod sanitization, credential hygiene	**26 / 26 PASSED**
`backend/phase1-test.js`	Schema extraction, catalog parsing, foreign key resolution	**55 / 55 PASSED**
`backend/phase2-test.js`	Health Engine rules R001 (Missing PK) and R002 (Orphaned Table)	**49 / 49 PASSED**
`backend/phase3-test.js`	Rules R003 (Missing FK), R004 (Index Review), and R005 (DAG Dependency)	**42 / 42 PASSED**
`backend/phase4-test.js`	Analysis summary aggregation and project-defined health score calculation	**59 / 59 PASSED**
`phase5-test.js`	Database health dashboard integration and live analysis verification	**51 / 51 PASSED**
`phase6-test.js`	SQL Console execution, SELECT/SHOW/DESCRIBE/EXPLAIN, Read-only rejection	**50 / 50 PASSED**
`phase7-test.js`	Full CRUD lifecycle (Create, Read, Update, Delete) on MySQL tables	**37 / 37 PASSED**
`e2e-audit-test.js`	Comprehensive end-to-end integration audit against `college_db`	**58 / 58 PASSED**
TypeScript Check	`pnpm exec tsc --noEmit`	**0 ERRORS**
Production Build	`pnpm build` (Next.js 16 Webpack bundle)	**SUCCESS (3.0s)**